

Introduction to Theoretical Computer Science

CS2490 - Session 10: DFA minimization – practice

Pierre Cagne

Fall 2026

September 8, 2026

Recall from last session

From any DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$, make a DFA $\mathcal{A}^{\min} = (\Sigma, Q^{\min}, \delta^{\min}, q_0^{\min}, F^{\min})$ as follows:

Recall from last session

From any DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$, make a DFA $\mathcal{A}^{\min} = (\Sigma, Q^{\min}, \delta^{\min}, q_0^{\min}, F^{\min})$ as follows:

- ▶ $\mathcal{A}$ must be trimmed (remove unreachable states otherwise), **No useless states.**

Recall from last session

From any DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$, make a DFA $\mathcal{A}^{\min} = (\Sigma, Q^{\min}, \delta^{\min}, q_0^{\min}, F^{\min})$ as follows:

- ▶ $\mathcal{A}$ must be trimmed (remove unreachable states otherwise), **No useless states.**
- ▶ $Q^{\min} = \{L_q \mid q \in Q\}$ is the set of residual languages, **Merge indistinguishable states.**

Recall from last session

From any DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$, make a DFA $\mathcal{A}^{\min} = (\Sigma, Q^{\min}, \delta^{\min}, q_0^{\min}, F^{\min})$ as follows:

- ▶ $\mathcal{A}$ must be trimmed (remove unreachable states otherwise), **No useless states.**
- ▶ $Q^{\min} = \{L_q \mid q \in Q\}$ is the set of residual languages, **Merge indistinguishable states.**
- ▶ $\delta^{\min}(a, L_q) = L_{\delta(a,q)}$ for any $q \in Q$ and $a \in \Sigma$, **Update transitions.**

Recall from last session

From any DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$, make a DFA $\mathcal{A}^{\min} = (\Sigma, Q^{\min}, \delta^{\min}, q_0^{\min}, F^{\min})$ as follows:

- ▶ $\mathcal{A}$ must be trimmed (remove unreachable states otherwise), **No useless states.**
- ▶ $Q^{\min} = \{L_q \mid q \in Q\}$ is the set of residual languages, **Merge indistinguishable states.**
- ▶ $\delta^{\min}(a, L_q) = L_{\delta(a,q)}$ for any $q \in Q$ and $a \in \Sigma$, **Update transitions.**
- ▶ $q_0^{\min} = L_{q_0}$, **Update initial state.**

Recall from last session

From any DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$, make a DFA $\mathcal{A}^{\min} = (\Sigma, Q^{\min}, \delta^{\min}, q_0^{\min}, F^{\min})$ as follows:

- ▶ $\mathcal{A}$ must be trimmed (remove unreachable states otherwise), No useless states.
- ▶ $Q^{\min} = \{L_q \mid q \in Q\}$ is the set of residual languages, Merge indistinguishable states.
- ▶ $\delta^{\min}(a, L_q) = L_{\delta(a,q)}$ for any $q \in Q$ and $a \in \Sigma$, Update transitions.
- ▶ $q_0^{\min} = L_{q_0}$, Update initial state.
- ▶ $F^{\min} = \{L_q \in Q^{\min} \mid q \in F\}$. Update accepting states.

Recall from last session

From any DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$, make a DFA $\mathcal{A}^{\min} = (\Sigma, Q^{\min}, \delta^{\min}, q_0^{\min}, F^{\min})$ as follows:

- ▶ $\mathcal{A}$ must be trimmed (remove unreachable states otherwise), No useless states.
- ▶ $Q^{\min} = \{L_q \mid q \in Q\}$ is the set of residual languages, Merge indistinguishable states.
- ▶ $\delta^{\min}(a, L_q) = L_{\delta(a,q)}$ for any $q \in Q$ and $a \in \Sigma$, Update transitions.
- ▶ $q_0^{\min} = L_{q_0}$, Update initial state.
- ▶ $F^{\min} = \{L_q \in Q^{\min} \mid q \in F\}$. Update accepting states.

By “update” is meant that whatever was before merging states remains.

Recall from last session

From any DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$, make a DFA $\mathcal{A}^{\min} = (\Sigma, Q^{\min}, \delta^{\min}, q_0^{\min}, F^{\min})$ as follows:

- ▶ $\mathcal{A}$ must be trimmed (remove unreachable states otherwise), No useless states.
- ▶ $Q^{\min} = \{L_q \mid q \in Q\}$ is the set of residual languages, Merge indistinguishable states.
- ▶ $\delta^{\min}(a, L_q) = L_{\delta(a,q)}$ for any $q \in Q$ and $a \in \Sigma$, Update transitions.
- ▶ $q_0^{\min} = L_{q_0}$, Update initial state.
- ▶ $F^{\min} = \{L_q \in Q^{\min} \mid q \in F\}$. Update accepting states.

By “update” is meant that whatever was before merging states remains.

Then, $\mathcal{A}^{\min}$ is minimal among all the DFA recognizing $L(\mathcal{A})$.

Let's prove it!

Suppose $\mathcal{A}' = (\Sigma, Q', \delta', q'_0, F')$ is another DFA recognizing $L(\mathcal{A})$.

Let's prove it!

Suppose $\mathcal{A}' = (\Sigma, Q', \delta', q'_0, F')$ is another DFA recognizing $L(\mathcal{A})$.

To prove that $\mathcal{A}'$ has **at least as many states** as $\mathcal{A}^{\min}$: pick a state q'_χ in $\mathcal{A}'$ for each state χ of $\mathcal{A}^{\min}$, and show that all the q'_χ are **distinct**.

Let's prove it!

Suppose $\mathcal{A}' = (\Sigma, Q', \delta', q'_0, F')$ is another DFA recognizing $L(\mathcal{A})$.

To prove that $\mathcal{A}'$ has **at least as many states** as $\mathcal{A}^{\min}$: pick a state q'_χ in $\mathcal{A}'$ for each state χ of $\mathcal{A}^{\min}$, and show that all the q'_χ are **distinct**.

Recall that each state χ in $\mathcal{A}^{\min}$ is **reachable**: write w_χ for a word that reaches it.

Let's prove it!

Suppose $\mathcal{A}' = (\Sigma, Q', \delta', q'_0, F')$ is another DFA recognizing $L(\mathcal{A})$.

To prove that $\mathcal{A}'$ has **at least as many states** as $\mathcal{A}^{\min}$: pick a state q'_χ in $\mathcal{A}'$ for each state χ of $\mathcal{A}^{\min}$, and show that all the q'_χ are **distinct**.

Recall that each state χ in $\mathcal{A}^{\min}$ is **reachable**: write w_χ for a word that reaches it. Then define $q'_\chi = \delta'^*(w_\chi, q'_0)$ as the state reached in $\mathcal{A}'$ by that same word.

Let's prove it!

Suppose $\mathcal{A}' = (\Sigma, Q', \delta', q'_0, F')$ is another DFA recognizing $L(\mathcal{A})$.

To prove that $\mathcal{A}'$ has **at least as many states** as $\mathcal{A}^{\min}$: pick a state q'_χ in $\mathcal{A}'$ for each state χ of $\mathcal{A}^{\min}$, and show that all the q'_χ are **distinct**.

Recall that each state χ in $\mathcal{A}^{\min}$ is **reachable**: write w_χ for a word that reaches it. Then define $q'_\chi = \delta'^*(w_\chi, q'_0)$ as the state reached in $\mathcal{A}'$ by that same word.

If there is χ and ψ such that $q'_\chi = q'_\psi$, then w_χ and w_ψ lead to the same state in $\mathcal{A}'$.

Let's prove it!

Suppose $\mathcal{A}' = (\Sigma, Q', \delta', q'_0, F')$ is another DFA recognizing $L(\mathcal{A})$.

To prove that $\mathcal{A}'$ has **at least as many states** as $\mathcal{A}^{\min}$: pick a state q'_χ in $\mathcal{A}'$ for each state χ of $\mathcal{A}^{\min}$, and show that all the q'_χ are **distinct**.

Recall that each state χ in $\mathcal{A}^{\min}$ is **reachable**: write w_χ for a word that reaches it. Then define $q'_\chi = \delta'^*(w_\chi, q'_0)$ as the state reached in $\mathcal{A}'$ by that same word.

If there is χ and ψ such that $q'_\chi = q'_\psi$, then w_χ and w_ψ lead to the same state in $\mathcal{A}'$. For any word u , $w_\chi \cdot u$ and $w_\psi \cdot u$ also lead to the same state in $\mathcal{A}'$ and thus are either **both accepted** or **both rejected** by $\mathcal{A}'$.

Let's prove it!

Suppose $\mathcal{A}' = (\Sigma, Q', \delta', q'_0, F')$ is another DFA recognizing $L(\mathcal{A})$.

To prove that $\mathcal{A}'$ has **at least as many states** as $\mathcal{A}^{\min}$: pick a state q'_χ in $\mathcal{A}'$ for each state χ of $\mathcal{A}^{\min}$, and show that all the q'_χ are **distinct**.

Recall that each state χ in $\mathcal{A}^{\min}$ is **reachable**: write w_χ for a word that reaches it. Then define $q'_\chi = \delta'^*(w_\chi, q'_0)$ as the state reached in $\mathcal{A}'$ by that same word.

If there is χ and ψ such that $q'_\chi = q'_\psi$, then w_χ and w_ψ lead to the same state in $\mathcal{A}'$. For any word u , $w_\chi \cdot u$ and $w_\psi \cdot u$ also lead to the same state in $\mathcal{A}'$ and thus are either **both accepted** or **both rejected** by $\mathcal{A}'$.

Since $\mathcal{A}^{\min}$ recognizes the same language, $w_\chi \cdot u$ and $w_\psi \cdot u$ also are either **both accepted** or **both rejected** by $\mathcal{A}^{\min}$.

Let's prove it!

Suppose $\mathcal{A}' = (\Sigma, Q', \delta', q'_0, F')$ is another DFA recognizing $L(\mathcal{A})$.

To prove that $\mathcal{A}'$ has **at least as many states** as $\mathcal{A}^{\min}$: pick a state q'_χ in $\mathcal{A}'$ for each state χ of $\mathcal{A}^{\min}$, and show that all the q'_χ are **distinct**.

Recall that each state χ in $\mathcal{A}^{\min}$ is **reachable**: write w_χ for a word that reaches it. Then define $q'_\chi = \delta'^*(w_\chi, q'_0)$ as the state reached in $\mathcal{A}'$ by that same word.

If there is χ and ψ such that $q'_\chi = q'_\psi$, then w_χ and w_ψ lead to the same state in $\mathcal{A}'$. For any word u , $w_\chi \cdot u$ and $w_\psi \cdot u$ also lead to the same state in $\mathcal{A}'$ and thus are either **both accepted** or **both rejected** by $\mathcal{A}'$.

Since $\mathcal{A}^{\min}$ recognizes the same language, $w_\chi \cdot u$ and $w_\psi \cdot u$ also are either **both accepted** or **both rejected** by $\mathcal{A}^{\min}$.

In other words, χ and ψ are **indistinguishable**. Since states in $\mathcal{A}^{\min}$ are pairwise distinguishable, it means that $\chi = \psi$.

Algorithm

Algorithm

- ▶ Start from a trimmed DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$.
No useless states.

Algorithm

- ▶ Start from a trimmed DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$.
No useless states.
- ▶ Build a table with all pairs of states (p, q) , $p, q \in Q$.
It will be symmetrical, so keeping only the upper half is enough.

Algorithm

- ▶ Start from a trimmed DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$.
No useless states.
- ▶ Build a table with all pairs of states (p, q) , $p, q \in Q$.
It will be symmetrical, so keeping only the upper half is enough.
- ▶ Mark pairs (p, q) where one is accepting and the other is not with a 0.
Pairs distinguishable by ε .

Algorithm

- ▶ Start from a trimmed DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$.
No useless states.
- ▶ Build a table with all pairs of states (p, q) , $p, q \in Q$.
It will be symmetrical, so keeping only the upper half is enough.
- ▶ Mark pairs (p, q) where one is accepting and the other is not with a 0.
Pairs distinguishable by ε .
- ▶ Propagate markings: if $\delta(a, p)$ and $\delta(a, q)$ is a pair marked k for some $a \in \Sigma$, then mark (p, q) with $k + 1$.
Pairs distinguishable by words of length $k + 1$.

Algorithm

- ▶ Start from a trimmed DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$.
No useless states.
- ▶ Build a table with all pairs of states (p, q) , $p, q \in Q$.
It will be symmetrical, so keeping only the upper half is enough.
- ▶ Mark pairs (p, q) where one is accepting and the other is not with a 0.
Pairs distinguishable by ε .
- ▶ Propagate markings: if $\delta(a, p)$ and $\delta(a, q)$ is a pair marked k for some $a \in \Sigma$, then mark (p, q) with $k + 1$.
Pairs distinguishable by words of length $k + 1$.
- ▶ Continue propagating until no new markings.
All distinguishable pairs are marked.

Algorithm

- ▶ Start from a trimmed DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$.
No useless states.
- ▶ Build a table with all pairs of states (p, q) , $p, q \in Q$.
It will be symmetrical, so keeping only the upper half is enough.
- ▶ Mark pairs (p, q) where one is accepting and the other is not with a 0.
Pairs distinguishable by ε .
- ▶ Propagate markings: if $\delta(a, p)$ and $\delta(a, q)$ is a pair marked k for some $a \in \Sigma$, then mark (p, q) with $k + 1$.
Pairs distinguishable by words of length $k + 1$.
- ▶ Continue propagating until no new markings.
All distinguishable pairs are marked.
- ▶ Unmarked pairs are indistinguishable: merge them.

Algorithm

- ▶ Start from a trimmed DFA $\mathcal{A} = (\Sigma, Q, \delta, q_0, F)$.
No useless states.
- ▶ Build a table with all pairs of states (p, q) , $p, q \in Q$.
It will be symmetrical, so keeping only the upper half is enough.
- ▶ Mark pairs (p, q) where one is accepting and the other is not with a 0.
Pairs distinguishable by ε .
- ▶ Propagate markings: if $\delta(a, p)$ and $\delta(a, q)$ is a pair marked k for some $a \in \Sigma$, then mark (p, q) with $k + 1$.
Pairs distinguishable by words of length $k + 1$.
- ▶ Continue propagating until no new markings.
All distinguishable pairs are marked.
- ▶ Unmarked pairs are indistinguishable: merge them.
- ▶ Update transitions, initial state and accepting states accordingly.

Algorithm: correctness

Claim: The states p and q can be distinguished by a word of length k if and only if the pair (p, q) is marked with a number $\leq k$.

Proof: by induction on k .

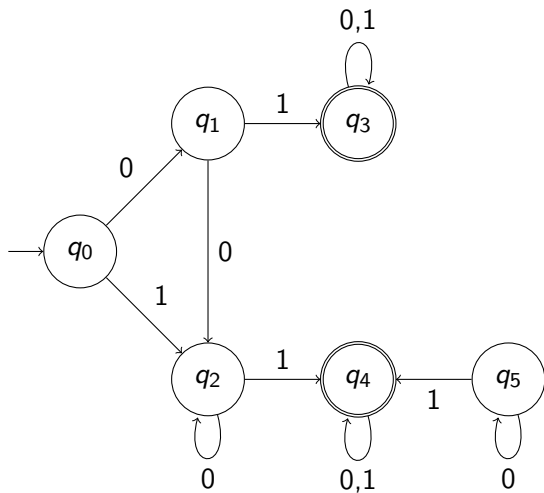
Algorithm: correctness

Claim: The states p and q can be distinguished by a word of length k if and only if the pair (p, q) is marked with a number $\leq k$.

Proof: by induction on k .

$\Rightarrow$ The algorithm results in $\mathcal{A}^{\min}$.

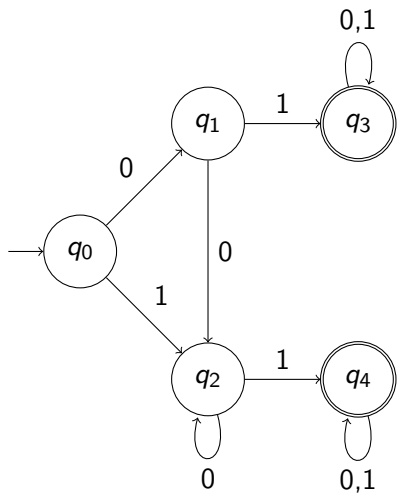
Algorithm: example



	q_0	q_1	q_2	q_3	q_4
q_0					
q_1					
q_2					
q_3					
q_4					

► Step 1: Trim and list all pairs.

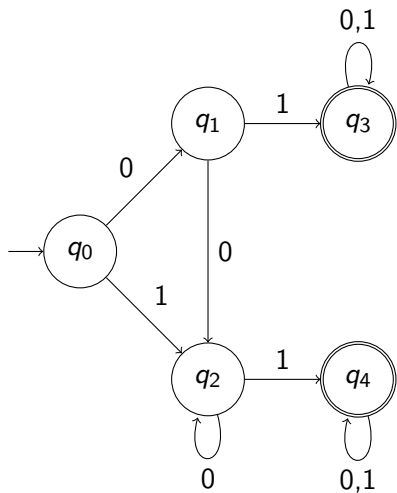
Algorithm: example



	q_0	q_1	q_2	q_3	q_4
q_0				0	0
q_1				0	0
q_2				0	0
q_3	0	0	0		
q_4	0	0	0		

- Step 2: Mark pairs (p, q) where one is accepting and the other is not (0).

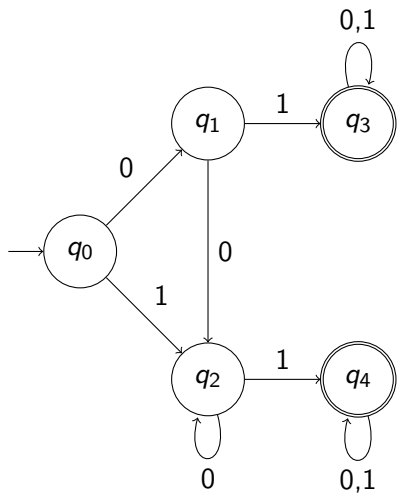
Algorithm: example



	q_0	q_1	q_2	q_3	q_4
q_0		1	1	0	0
q_1	1			0	0
q_2	1			0	0
q_3	0	0	0		
q_4	0	0	0		

- Step 3: Propagate markings (1) if transitions lead to marked pairs.

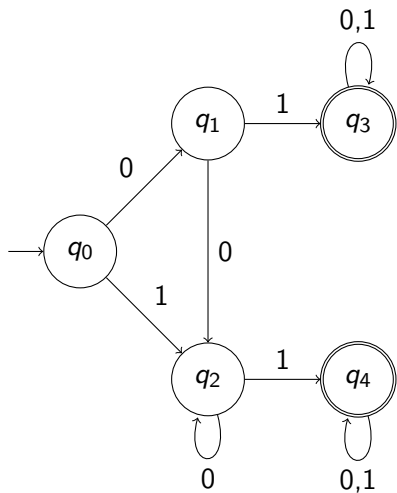
Algorithm: example



	q_0	q_1	q_2	q_3	q_4
q_0		1	1	0	0
q_1	1			0	0
q_2	1			0	0
q_3	0	0	0		
q_4	0	0	0		

- Step 4: Continue propagating until no new markings.

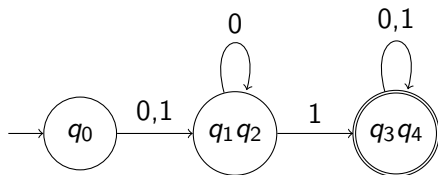
Algorithm: example



	q_0	q_1	q_2	q_3	q_4
q_0	$\times$	1	1	0	0
q_1	1	$\times$	$\checkmark$	0	0
q_2	1	$\checkmark$	$\times$	0	0
q_3	0	0	0	$\times$	$\checkmark$
q_4	0	0	0	$\checkmark$	$\times$

- Step 5: Unmarked pairs are indistinguishable, merge them.

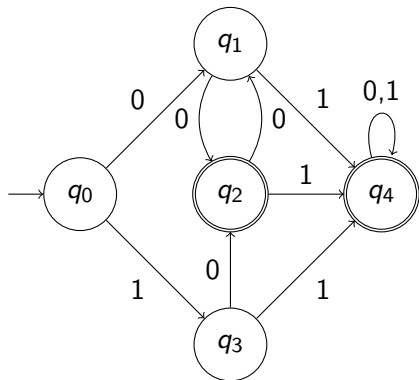
Algorithm: example



	q_0	q_1	q_2	q_3	q_4
q_0	X	1	1	0	0
q_1	1	X	✓	0	0
q_2	1	✓	X	0	0
q_3	0	0	0	X	✓
q_4	0	0	0	✓	X

- Step 5: Unmarked pairs are indistinguishable, merge them.

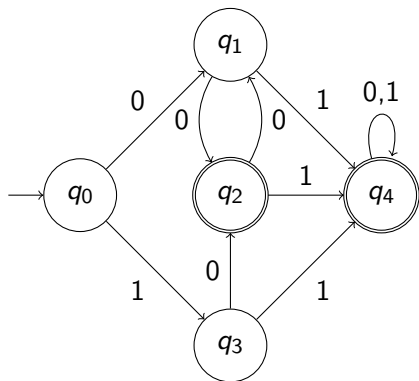
More practice



	q_0	q_1	q_2	q_3	q_4
q_0					
q_1					
q_2					
q_3					
q_4					

- Step 1: Trim and list all pairs.

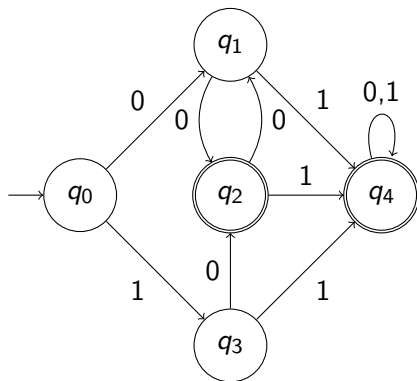
More practice



	q_0	q_1	q_2	q_3	q_4
q_0	X		0		0
q_1	X	X	0		0
q_2	X	X	X	0	
q_3	X	X	X	X	0
q_4	X	X	X	X	X

- Step 2: Mark pairs (p, q) where one is accepting and the other is not (0).

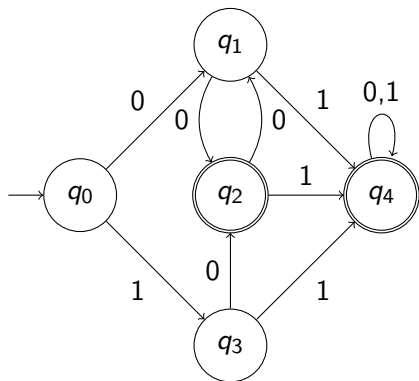
More practice



	q_0	q_1	q_2	q_3	q_4
q_0	X	1	0	1	0
q_1	X	X	0		0
q_2	X	X	X	0	1
q_3	X	X	X	X	0
q_4	X	X	X	X	X

- Step 3: Propagate markings (1) if transitions lead to marked pairs.

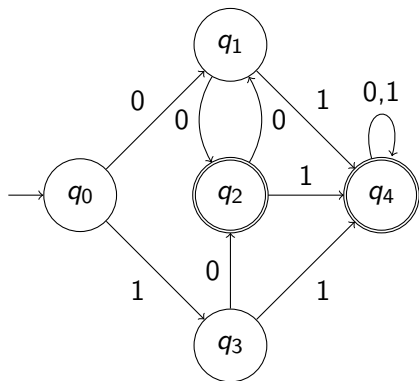
More practice



	q_0	q_1	q_2	q_3	q_4
q_0	X	1	0	1	0
q_1	X	X	0		0
q_2	X	X	X	0	1
q_3	X	X	X	X	0
q_4	X	X	X	X	X

- Step 4: Continue propagating until no new markings.

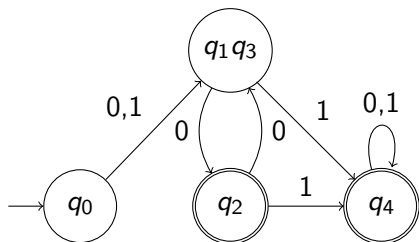
More practice



	q_0	q_1	q_2	q_3	q_4
q_0	X	1	0	1	0
q_1	X	X	0	✓	0
q_2	X	X	X	0	1
q_3	X	X	X	X	0
q_4	X	X	X	X	X

- Step 5: Unmarked pairs are indistinguishable, merge them.

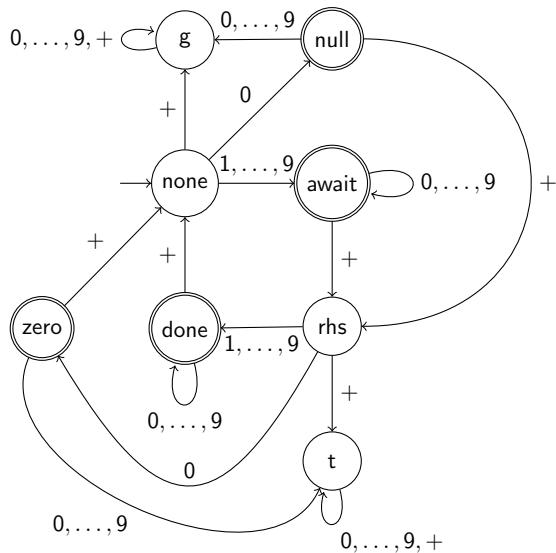
More practice



	q_0	q_1	q_2	q_3	q_4
q_0	X	1	0	1	0
q_1	X	X	0	✓	0
q_2	X	X	X	0	1
q_3	X	X	X	X	0
q_4	X	X	X	X	X

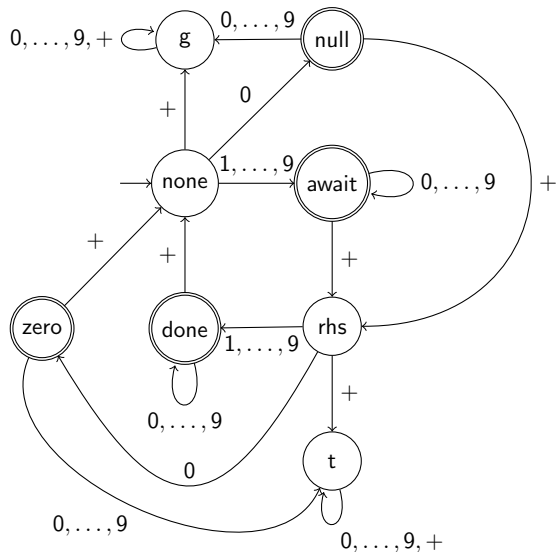
- Step 5: Unmarked pairs are indistinguishable, merge them.

More practice (cont'd)



► Step 1: Trim and list all pairs.

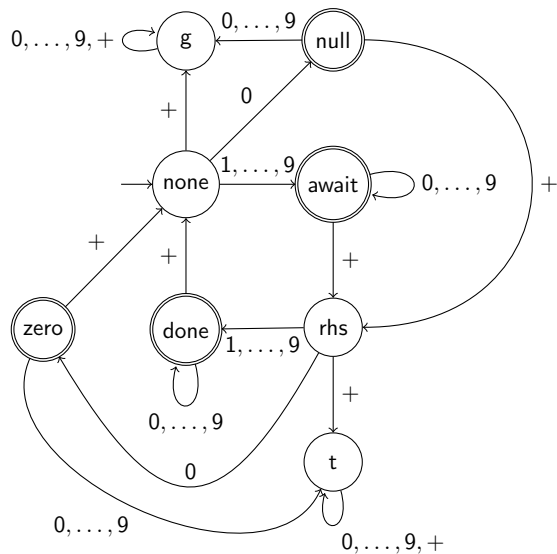
More practice (cont'd)



	none	await	done	rhs	g	null	zero	t
none	X	0	0			0	0	
await	X	X		0	0			0
done	X	X	X	0	0			0
rhs	X	X	X	X		0	0	
g	X	X	X	X	X	0	0	
null	X	X	X	X	X	X		0
zero	X	X	X	X	X	X	X	0
t	X	X	X	X	X	X	X	X

- Step 2: Mark pairs (p, q) where one is accepting and the other is not (0).

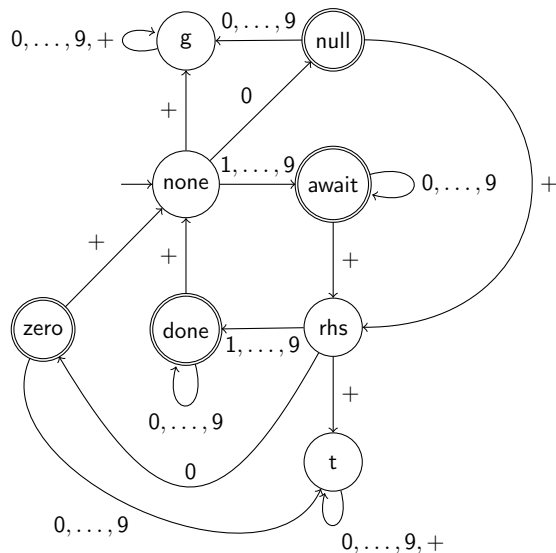
More practice (cont'd)



	none	await	done	rhs	g	null	zero	t
none	X	0	0		1	0	0	1
await	X	X		0	0	1	1	0
done	X	X	X	0	0	1	1	0
rhs	X	X	X	X	1	0	0	1
g	X	X	X	X	X	0	0	
null	X	X	X	X	X	X		0
zero	X	X	X	X	X	X	X	0
t	X	X	X	X	X	X	X	X

- Step 3: Propagate markings (1) if transitions lead to marked pairs.

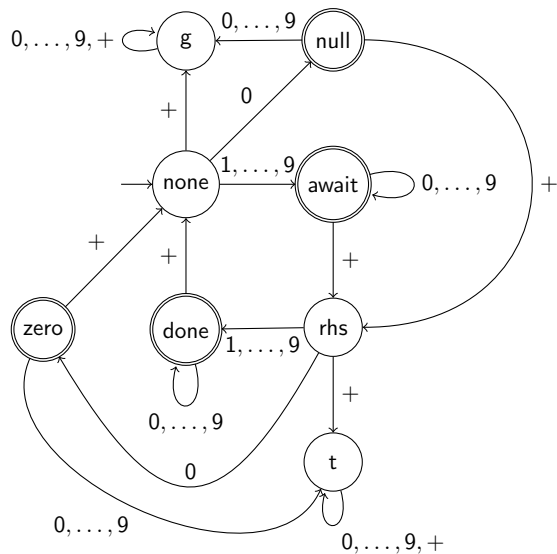
More practice (cont'd)



	none	await	done	rhs	g	null	zero	t
none	X	0	0		1	0	0	1
await	X	X		0	0	1	1	0
done	X	X	X	0	0	1	1	0
rhs	X	X	X	X	1	0	0	1
g	X	X	X	X	X	0	0	
null	X	X	X	X	X	X		0
zero	X	X	X	X	X	X	X	0
t	X	X	X	X	X	X	X	X

- Step 4: Continue propagating until no new markings.

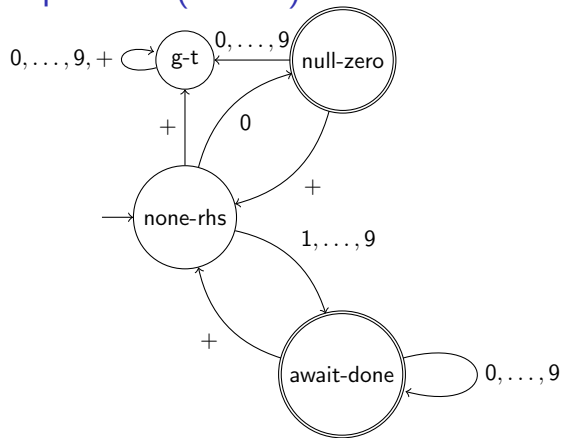
More practice (cont'd)



	none	await	done	rhs	g	null	zero	t
none	X	0	0	✓	1	0	0	1
await	X	X	✓	0	0	1	1	0
done	X	X	X	0	0	1	1	0
rhs	X	X	X	X	1	0	0	1
g	X	X	X	X	X	0	0	✓
null	X	X	X	X	X	X	✓	0
zero	X	X	X	X	X	X	X	0
t	X	X	X	X	X	X	X	X

- Step 5: Unmarked pairs are indistinguishable, merge them.

More practice (cont'd)



	none	await	done	rhs	g	null	zero	t
none	X	0	0	✓	1	0	0	1
await	X	X	✓	0	0	1	1	0
done	X	X	X	0	0	1	1	0
rhs	X	X	X	X	1	0	0	1
g	X	X	X	X	X	0	0	✓
null	X	X	X	X	X	X	✓	0
zero	X	X	X	X	X	X	X	0
t	X	X	X	X	X	X	X	X

- Step 5: Unmarked pairs are indistinguishable, merge them.

A non obvious question

Consider the two following languages over the alphabet $\Sigma = \{a, b\}$:

A non obvious question

Consider the two following languages over the alphabet $\Sigma = \{a, b\}$:

► $L_1 = \{w \in \Sigma^* \mid |w| \geq 1\}$

A non obvious question

Consider the two following languages over the alphabet $\Sigma = \{a, b\}$:

- ▶ $L_1 = \{w \in \Sigma^* \mid |w| \geq 1\}$
- ▶ $L_2 = \{a^n \mid n \geq 1\} \cup \{a^k b a^{k_1} b a^{k_2} \dots b a^{k_m} \mid k \geq 0, m \geq 1, \text{ each } k_i \geq 0\}$

A non obvious question

Consider the two following languages over the alphabet $\Sigma = \{a, b\}$:

- ▶ $L_1 = \{w \in \Sigma^* \mid |w| \geq 1\}$
- ▶ $L_2 = \{a^n \mid n \geq 1\} \cup \{a^k b a^{k_1} b a^{k_2} \dots b a^{k_m} \mid k \geq 0, m \geq 1, \text{ each } k_i \geq 0\}$

Claim: They are the same language!

A non obvious question

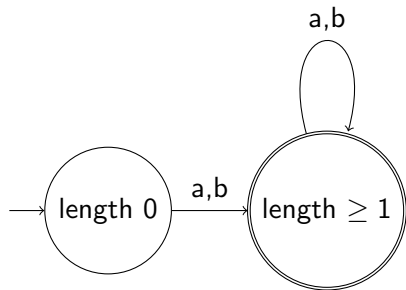
Consider the two following languages over the alphabet $\Sigma = \{a, b\}$:

- ▶ $L_1 = \{w \in \Sigma^* \mid |w| \geq 1\}$
- ▶ $L_2 = \{a^n \mid n \geq 1\} \cup \{a^k b a^{k_1} b a^{k_2} \dots b a^{k_m} \mid k \geq 0, m \geq 1, \text{ each } k_i \geq 0\}$

Claim: They are the same language!

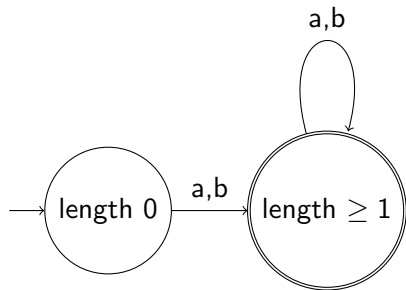
What strategy can you employ to prove this?

A non obvious question (cont'd)

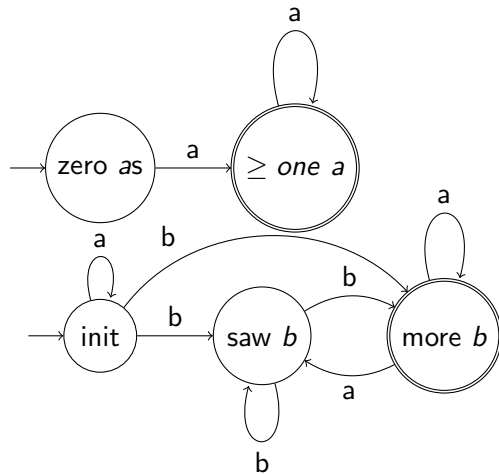


This DFA recognizes L_1 .

A non obvious question (cont'd)



This DFA recognizes L_1 .



This NFA recognizes L_2 .

A non obvious question (cont'd)

A non obvious question (cont'd)

- ▶ We can minimize the DFA for L_1 (it is already minimal).

A non obvious question (cont'd)

- ▶ We can minimize the DFA for L_1 (it is already minimal).
- ▶ We can convert the NFA for L_2 to a DFA.

A non obvious question (cont'd)

- ▶ We can minimize the DFA for L_1 (it is already minimal).
- ▶ We can convert the NFA for L_2 to a DFA.
- ▶ We can minimize the resulting DFA for L_2 .

A non obvious question (cont'd)

- ▶ We can minimize the DFA for L_1 (it is already minimal).
- ▶ We can convert the NFA for L_2 to a DFA.
- ▶ We can minimize the resulting DFA for L_2 .
- ▶ If the two minimal DFAs are identical, then $L_1 = L_2$.

A non obvious question (cont'd)

- ▶ We can minimize the DFA for L_1 (it is already minimal).
- ▶ We can convert the NFA for L_2 to a DFA.
- ▶ We can minimize the resulting DFA for L_2 .
- ▶ If the two minimal DFAs are identical, then $L_1 = L_2$.

Let's see it in action!